

AIDA vs hosted SaaS PM (Linear / Jira / GitHub Projects)

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — pricing and feature comparisons should be re-verified against [docs/competitive-analysis/marketplace-roster.md](#) (the current 2026-07-07 roster) before any procurement-facing use.

The TL;DR: **AIDA is not trying to replace your project-management suite.** Linear, Jira, and GitHub Projects do things AIDA explicitly doesn't try to do — multi-team coordination, customer-facing roadmaps, SLAs, sprint reporting, OKR alignment. AIDA's pitch is the *intent-graph layer that lives in the repo*. You'll often want both, with each owning a different scope of the same work.

What each tool is actually for

Tool	Native scope	Where it lives	Cost shape
AIDA	Requirement graph + code-to-spec traceability + MCP-native context for agents	The repo (orphan-branch YAML + local SQLite cache); no server required for solo use	Free, self-hosted. Optional <code>aida-server</code> adds REST + gRPC + dashboard but isn't required
Linear	Issue tracking for engineering teams; opinionated workflow + fast UI + agent integrations	Hosted SaaS	Free up to 250 issues; Basic ~\$10/user/mo, Business ~\$16/user/mo (verify against vendor pricing)
Jira	Enterprise issue/project tracking; configurable workflows; Rovo agents GA May 2026 (Rovo "Max" multistep in Early Access)	Hosted SaaS (Cloud) or self-hosted (Data Center)	Standard / Premium / Enterprise tiers ~\$8-16/user/mo and up (verify against vendor pricing)

Tool	Native scope	Where it lives	Cost shape
GitHub Projects	Lightweight, GitHub-native boards/tables/roadmaps tied to issues + PRs	Hosted alongside the repo on GitHub	Included with GitHub plans

These are not the same shape of thing. AIDA is a **database with primitives optimized for AI consumption**; Linear/Jira/GitHub Projects are **workflow products with UIs optimized for human teams**.

What AIDA does that hosted SaaS PM doesn't

- **Code-to-spec trace comments.** `// trace:FR-1-042 | ai:claude` in source files, walked by `aida trace`. Linear's LIN-123 keys appear in commit messages; nothing enforces that the code still implements the issue when both move. AIDA's traceability is a checkable invariant.
- **MCP server.** `aida mcp-serve` exposes the requirement graph as native Claude Code tools over JSON-RPC stdio. Shortcut has an MCP server (the best-in-class among PM tools as of 2026-03-17); Linear and Jira don't ship one natively (community MCP bridges exist with varying maintenance). Cursor / Windsurf / Claude Code can call AIDA's MCP tools directly. *Note: vendor MCP support is changing quickly — re-verify before relying on this claim.*
- **Git-canonical storage.** The orphan `aida-store` branch is the writer of record. The graph travels with the repo; clone the repo and you have the requirements. No SaaS dependency, no vendor lock-in, no separate backup story.
- **Zero per-user cost.** Linear at 5 developers × \$10/user/mo is \$600/year. Jira at 50 developers is in the \$5K-10K range. AIDA is free and self-hosted; there is no "user" billing axis.
- **Stable IDs across forks/clones.** Distributed identity (FR-1-052 → promoted to FR-052 at merge gate) means two clones can issue IDs offline without colliding. SaaS PM owns the ID namespace centrally; offline work has to be re-keyed.

These all derive from the same root: AIDA's intent graph is *in your repo, queryable by your tools*, and *checkable against your code*. That's a fundamentally different shape than hosted SaaS.

What hosted SaaS PM does that AIDA doesn't (and doesn't try to)

- **Polished UIs.** Linear in particular has a UI that's hard to match without a real product team. AIDA's React dashboard (port 5173) covers the daily-driver kanban/queue use case; it doesn't compete with Linear on visual polish.

- **Cross-team coordination workflows.** Sprints across multiple teams, dependency-tracking across projects, executive roadmap views, OKR alignment. AIDA's primitives can model these, but the *workflow product* is what teams pay for.
- **Customer-facing surfaces.** Public roadmaps, feature voting, issue intake from external users. Out of AIDA's scope by design.
- **SLA and incident workflows.** PagerDuty integration, on-call rotations, status-page wiring. Not AIDA's problem space.
- **Reporting and analytics.** Velocity charts, burndown charts, cycle-time dashboards. AIDA has `aida history` and `aida analytics`, but they're for individual project introspection, not for portfolio-level reporting to leadership.
- **Permissioning.** Linear/Jira have rich org/team/role permission models. AIDA's model is *whoever can write to the repo can write to the graph* — appropriate for solo developers and small trusted teams, insufficient for enterprises with regulatory access control.

If the team needs any of these things, *use the SaaS tool for that thing*. AIDA doesn't compete in this slot.

Composition: how AIDA + hosted SaaS PM actually compose

The realistic deployment is **AIDA as the in-repo intent graph; SaaS PM as the human-coordination layer**. The seam runs along this fault line:

Layer	Owner	Examples
Customer-facing intake	Linear/Jira	Bug reports, feature requests, support escalations
Sprint planning / standups	Linear/Jira/GitHub Projects	Velocity, capacity, board grooming
Engineering decisions (the "why")	AIDA	ADR-style decision records, design rationale, scenario docs
Code-to-spec traceability	AIDA	// trace: comments, aida trace, merge-gate checks
Agent context	AIDA via MCP	<code>list_requirements</code> , <code>show_requirement</code> , <code>aida://requirements/tree</code>
Cross-project portfolio reporting	SaaS PM	Quarterly reviews, leadership dashboards

Two complementary patterns:

1. **Bidirectional sync** (heavyweight). Mirror a subset of Linear/Jira issues into AIDA as functional requirements, link by ID. Costs an integration to build + maintain; pays for itself when the team genuinely needs both axes (e.g., regulatory traceability + standard PM workflow).
2. **One-way refs** (lightweight, recommended default). Issues live in Linear/Jira; AIDA requirements reference them via URL or custom-field IDs. `aida add --tags`

linear:LIN-1234 is enough for “I know this AIDA req is the engineering side of that Linear ticket.”

The one-way pattern composes cleanly with `aida-gitlab` / `aida-github` / `aida-jira` integrations — those exist to push references in one direction, not to be the source of truth for both.

When to deploy each

Rough decision tree:

1. “Solo developer or small trusted team, AI-assisted, want context in the repo?” → **AIDA, no SaaS PM needed.**
 2. “Multi-team org with sprint planning + customer-facing roadmap needs?” → **SaaS PM is required**; add AIDA for the intent-graph + code-traceability layer the SaaS tool can’t provide.
 3. “Enterprise with regulatory traceability requirements (ISO 26262, DO-178C, IEC 62304)?” → **IBM DOORS or similar is required**; AIDA is too lightweight for formal certification audits. AIDA composes alongside DOORS for the AI-context layer.
 4. “Open-source project with public issue tracker?” → **GitHub Issues + AIDA**. Issues for external contributors; AIDA for the internal intent graph.
 5. “AI-first project where agents are the primary consumer of project context?” → **AIDA is the primary; SaaS PM is optional**. This is the case AIDA was built for.
-

Honest scope statement

AIDA’s pitch is **not** “cheaper Linear” or “self-hosted Jira.” If a team would otherwise pay for Linear/Jira and gets a useful product from them, AIDA does not replace that product. AIDA replaces the *gap between Linear/Jira and the code* — the thing nobody has historically owned, that becomes acutely visible the moment AI agents start writing code that should trace to intent.

The right deployment in a team that already has Linear is “*keep Linear, add AIDA underneath.*” The deployment in a solo project that doesn’t have a PM tool is “*use AIDA, skip Linear.*” Picking one against the other in a binary way is almost always the wrong framing.

See also

- [competitive-analysis/2026-03-17-landscape-scan.md](#) — the broader landscape scan (2026-03-17), including specific pricing snapshots.
- [vs-karpathy-md.md](#) — when even hosted PM is overkill and structured markdown is enough.
- [vs-ultrareview.md](#) — review-tool comparison (orthogonal but related).